

DisloCluster User Manual

Installation, Build, and Simulation Guide
Coupled Cluster Dynamics – Dislocation Dynamics of Irradiated Zirconium

Nasr M. Ghoniem

August 7, 2026

Repository: <https://github.com/Ghoniem/DisloCluster>

Abstract

DisloCluster simulates radiation effects in zirconium by coupling a zero-dimensional (0-D) reduced cluster-dynamics model of point-defect and dislocation-loop evolution (ZrMicro) to a three-dimensional, spatially resolved cluster-dynamics / dislocation-dynamics code (MoDELib3, a fork of MoDELib2-NNL). This manual takes a user from a fresh clone of the repository on a new Windows machine to a completed simulation: installing the prerequisites (Section 2), creating the Python virtual environment (Section 3), compiling the two C++ codes (Section 4), and running the three kinds of simulation the repository supports — the standalone 3-D single-crystal test (Section 5), the 0-D ZrMicro model (Section 6), and the coupled 0-D $\leftrightarrow$ 3-D dose march (Section 7). Section 8 explains where to change physical and numerical parameters, Section 9 collects the known failure modes and their fixes, and Section 10 describes the provenance system that makes every run reproducible. Appendix A is a condensed architecture reference, Appendix B summarizes the physics added relative to the upstream MoDELib2-NNL code, and Appendix C is a one-page command summary.

Contents

1	Introduction	3
1.1	What DisloCluster does	3
1.2	The three ways to run it	3
1.3	Repository layout	3
1.4	Conventions used in this manual	4
2	Installation	5
2.1	Prerequisites	5
2.2	Step 1: Windows tools	5
2.3	Step 2: clone the repository	5
2.4	Step 3: WSL and Ubuntu	6
2.5	Step 4: SUNDIALS 7.1.1	6
3	The Python virtual environment	6
3.1	Create the venv and install dependencies	6
3.2	Register the Jupyter kernel	7
3.3	Verify path resolution	7

4	Building the two codes	7
4.1	The 0-D ZrMicro solver (Windows, native)	7
4.2	The 3-D MoDELib3 code (inside WSL)	7
4.3	Verify the builds	8
5	First simulation: the 3-D single-crystal test	8
5.1	The case	8
5.2	Quick start: figures in minutes	9
5.3	Running the solve	9
5.4	Reading the figures critically	10
6	The 0-D ZrMicro simulation	10
6.1	Running it	10
6.2	Inputs: the Excel workbook	10
6.3	The state vector	11
6.4	Solver configuration	11
7	The coupled 0-D $\leftrightarrow$ 3-D simulation	11
7.1	The idea	11
7.2	The notebook's three control blocks	12
7.3	Meshing, and the materialized simulation directory	13
7.4	Fast-solve backends	13
7.5	Running the march	13
7.6	Outputs	14
8	Changing things	14
9	Troubleshooting	15
10	Reproducibility and provenance	16
A	Architecture reference	17
A.1	The governing split	17
A.2	Key components	17
A.3	The three interfaces	17
A.4	Unit systems	17
A.5	Call graph	19
A.6	Known non-correspondences between the 0-D and 3-D models	19
B	Changes relative to upstream MoDELib2-NNL	19
C	Quick reference	20

1 Introduction

1.1 What DisloCluster does

Irradiated zirconium develops a microstructure of prismatic $\langle a \rangle$ interstitial loops and basal $\langle c \rangle$ vacancy loops, fed by the point defects (vacancies, single interstitials, di- and tri-interstitials) that displacement cascades produce. The physics splits into two widely separated time scales: the *mobile* point-defect species relax to their local sink environment in milliseconds, while the *immobile* loop populations evolve over dose steps of order 10^7 s. DisloCluster exploits that split with an operator-split quasi-steady-state (QSSA) scheme: within each dose step the fast mobile species are solved as a steady reaction–diffusion field, and the slow loop populations are then marched in dose against the frozen mobile field.

The repository contains two codes that realize this physics at different levels of spatial resolution, plus the coupling layer between them:

- **ZrMicro** (`ZrClusterDynamics/ZrMicro/`) — the 0-D reduced model: 19 coupled ODEs (12 physical species, 6 conservation accumulators, and the evolving network dislocation density ρ_N), integrated with SUNDIALS CVODE through a compiled C++ solver, orchestrated and post-processed in Python.
- **MoDELib3** (`MoDELib3/`) — the 3-D spatially resolved cluster-dynamics / dislocation-dynamics code: a finite-element solver for the steady mobile boundary-value problem and a pointwise dose march for the four loop families, run by the `DDomp` executable.
- **The coupling layer** (`ZrMicro/py_utils/modelib/*.py` and the notebook `coupled_0d_3d_ZrMicro.ipynb`) — exchanges the loop sink field and the steady mobile field between the two codes over a shared dose schedule.

1.2 The three ways to run it

Everything in this manual funnels into one of three simulation types, and it is worth fixing them in mind now:

1. **The 3-D single-crystal run** (Section 5) — a standalone MoDELib3 simulation of a $1\ \mu\text{m}$ cubic grain under irradiation, driven by a shell script. No Python is involved in the solve. This is the recommended first test on a new machine, because the repository ships with the results of a completed solve (`ev1/`), so the post-processing can be exercised in minutes, before the hours-long solve is attempted.
2. **The 0-D ZrMicro run** (Section 6) — the reduced model alone, driven by the notebook `ZrMicro.ipynb`. Fast (minutes), and the source of the fitted parameter set.
3. **The coupled 0-D $\leftrightarrow$ 3-D march** (Section 7) — the operator-split coupling, driven by the notebook `coupled_0d_3d_ZrMicro.ipynb`. Python owns the dose loop, MoDELib3 solves the fast mobile field, and the ZrMicro C++ integrator marches the immobile state at every quadrature point.

1.3 Repository layout

The repository is self-contained: nothing outside the repository root is required except the Python interpreter, SUNDIALS, and (on Windows) WSL. The layout is:

```

DisloCluster/          <- repository root (.dislocluster_root marker)
|- .DisloClusterVenv/   Python 3.14 environment (you create this)
|- Libraries/           SUNDIALS install location (you create this)
| \- sundials-7.1.1/
|- requirements.txt      pinned Python dependencies
|- Docs/
| |- DisloCluster Manual/ this manual
| \- Formulation/        formulation documents + build_modelib_wsl.sh
|- ZrClusterDynamics/   0-D side
| |- Gmsh/               mesh generator (generate_mesh.py) + meshes/
| \- ZrMicro/
|   |- code/              notebooks + solver.cpp
|   | |- coupled_0d_3d_ZrMicro.ipynb <- coupled 0-D <-> 3-D driver
|   | |- ZrMicro.ipynb      0-D run + parameter identification
|   | \- coupling_demo.ipynb minimal coupling walk-through
|   |- cpp_utils/         C++ RHS + CMake build (SUNDIALS CVODE)
|   |- py_utils/          Python model chain; paths.py lives here
|   |- input/             Excel parameter workbook
|   |- build/              compiled solver.exe (created by the build)
|   \- output/            timestamped run directories
\-- MoDELib3/            3-D side (fork of MoDELib2-NNL)
    |- Library/Materials/  Zr3d_ghoniem.txt is the coupled material
    |- tutorials/zrmicro_coupled/ the single-crystal verification case
    \- build/              DDomp + pyMoDELib (built inside WSL)

```

Nothing in the repository hard-codes an absolute path. The single source of truth for every location is the module

`ZrClusterDynamics/ZrMicro/py_utils/paths.py`

which finds the repository root by walking up to the `.dislocluster_root` marker, so the repository can be cloned or moved anywhere. Three optional environment variables override the defaults: `DISLOCLUSTER_ROOT` (repository root), `MODELIB_ROOT` (a `MoDELib3` checkout outside the tree), and `MODELIB_BUILD` (its build directory).

Note. Some in-repository documents (`README.md`, the `CLAUDE.md` files, and the docstring of `paths.py`) still refer to the formulation documents and the WSL build script as living under `ZrClusterDynamics/Docs/`. In the current tree that folder is `Docs/` at the *repository root*. This manual uses the actual locations throughout; where the difference matters (the build script's default arguments, Section 4.2) it is called out explicitly.

1.4 Conventions used in this manual

The primary platform is Windows 11 with WSL 2 (Ubuntu): `ZrMicro` builds and runs natively on Windows, while `MoDELib3` is built and executed inside WSL, because it needs a GNU toolchain, Eigen, FFTW and SuiteSparse. Command blocks are colour-coded:

```
PS> a Windows PowerShell command, run from the repository root
```

```
$ a bash command inside WSL (or any Linux shell)
```

On a native Linux or macOS machine the same steps apply with the obvious simplifications: there is no WSL boundary, both codes build in the same shell, the venv lives at `.DisloClusterVenv/bin/`, and `paths.py` automatically selects the native binaries.

2 Installation

2.1 Prerequisites

Table 1 lists everything the repository needs. The rest of this section walks through installing each item in order.

Table 1: Prerequisites. Windows-side tools build and run ZrMicro; WSL-side tools build and run MoDELlib3.

component	version	side	purpose
Git	any recent	Windows	clone the repository
Python	3.14 (python.org)	Windows	venv, notebooks, orchestration
VS Build Tools	2022 (C++ workload)	Windows	compile ZrMicro
CMake	≥ 3.16	Windows	configure ZrMicro and SUNDIALS
SUNDIALS	7.1.1	Windows	CVODE/ARKODE for ZrMicro
WSL 2 + Ubuntu	22.04 or 24.04	WSL	build and run MoDELlib3
apt packages	Sec. 4.2	WSL	installed by the build script

Warning. Do not use Anaconda Python for this project. Its NumPy 1.x/2.x mix conflicts with the SciPy pinned in `requirements.txt`. Install CPython 3.14 from <https://www.python.org> and build the venv from that interpreter.

2.2 Step 1: Windows tools

1. **Git for Windows** — <https://git-scm.com>. Default options are fine.
2. **Python 3.14** — <https://www.python.org/downloads/>. Install for all users (the manual assumes `C:\Python314\python.exe`; adjust paths if you install elsewhere). It is not necessary to add it to PATH — every command in this manual calls an interpreter by explicit path.
3. **Visual Studio 2022 Build Tools** — <https://visualstudio.microsoft.com/downloads/>, workload “Desktop development with C++”. This provides the MSVC compiler and the OpenMP runtime that ZrMicro’s parallel batch mode uses.
4. **CMake** — <https://cmake.org/download/>, and let the installer add it to PATH. Any CMake ≥ 3.16 works for the Windows-side builds.

2.3 Step 2: clone the repository

From the directory that should contain the repository (any location works; no path is hard-coded):

```
PS> git clone https://github.com/Ghoniem/DisloCluster.git
PS> cd DisloCluster
```

The repository is private, so the clone authenticates with your GitHub account (Git for Windows will pop up a browser-based credential prompt the first time).

All subsequent commands in this manual are issued *from the repository root* unless stated otherwise.

Note. DisloCluster was assembled from two previously separate repositories, and ZrClusterDynamics/ and MoDELib3/ may still carry their own .git directories in some checkouts. The root marker file .dislocluster_root — not any .git — is what identifies the repository root to paths.py, and paths.git_hash() falls back through root → ZrClusterDynamics → MoDELib3 so that provenance tags stay meaningful either way.

2.4 Step 3: WSL and Ubuntu

MoDELib3 needs a Linux toolchain. On Windows this is provided by WSL2:

```
PS> wsl --install
```

Reboot when prompted, then complete the Ubuntu first-run setup (create a Linux username and password). Verify:

```
PS> wsl -e bash -c "lsb_release -d && gcc --version | head -1"
```

The compilers and libraries MoDELib3 needs inside WSL are installed automatically by the build script in Section 4.2; nothing further is required here.

2.5 Step 4: SUNDIALS 7.1.1

ZrMicro's C++ solver links against SUNDIALS 7.1.1 (modules cvode, arkode, nvecserial, sunlinsolband, sunlinsolspgmr). The ZrMicro CMake build looks for it in two places, in order: <repo>/Libraries/sundials-7.1.1/ (preferred), then the system-wide CMake package registry (e.g. an install under C:\Program Files (x86)\SUNDIALS).

To build and install it into the repository tree (recommended — it keeps the whole project self-contained), run from a scratch directory:

```
PS> curl.exe -LO https://github.com/LLNL/sundials/releases/download/v7.1.1/sundials-7.1.1.tar.gz
PS> tar -xzf sundials-7.1.1.tar.gz
PS> cmake -S sundials-7.1.1 -B sundials-build '
    -DCMAKE_INSTALL_PREFIX=<path-to>\DisloCluster\Libraries\sundials-7.1.1" '
    -DBUILD_SHARED_LIBS=OFF -DBUILD_STATIC_LIBS=ON '
    -DEXAMPLES_ENABLE_C=OFF -DEXAMPLES_INSTALL=OFF
PS> cmake --build sundials-build --config Release --target install
```

Replace <path-to> with the actual location of your clone. The install step populates Libraries/sundials-7.1.1/ with include/, lib/ and the CMake package files that find_package(SUNDIALS) consumes.

3 The Python virtual environment

3.1 Create the venv and install dependencies

The whole repository shares one virtual environment, .DisloClusterVenv/, at the repository root. From the repository root:

```
PS> C:\Python314\python.exe -m venv .DisloClusterVenv
PS> .DisloClusterVenv\Scripts\python.exe -m pip install -r requirements.txt
```

requirements.txt pins the full stack — NumPy 2.4, SciPy 1.17, pandas, Matplotlib, openpyxl (the Excel workbook reader), the Gmsh SDK (mesh generation for the coupled notebook), and the complete JupyterLab toolchain — so the install is reproducible on any machine.

3.2 Register the Jupyter kernel

The notebooks expect a kernel named `dislocluster`:

```
PS> .DisloClusterVenv\Scripts\python.exe -m ipykernel install --user '
      --name dislocluster --display-name "Python 3.14 (DisloCluster)"
```

When a notebook is opened in JupyterLab or VS Code, select the kernel “Python 3.14 (DisloCluster)”.

3.3 Verify path resolution

Before building anything, confirm that the path-resolution layer sees your checkout correctly:

```
PS> .DisloClusterVenv\Scripts\python.exe ZrClusterDynamics\ZrMicro\py_utils\paths.py
```

This prints one OK/MISS line per resolved location (repository root, venv, 0-D code, solver binary, MoDELib3 root and build, material file, coupled simulation case, ...). At this stage MISS is expected for the two solver binaries — they have not been built yet — and everything else should read OK. Re-run this command whenever something cannot be found; it is the fastest diagnostic in the repository.

4 Building the two codes

4.1 The 0-D ZrMicro solver (Windows, native)

From the repository root:

```
PS> cmake -S ZrClusterDynamics\ZrMicro\cpp_utils -B ZrClusterDynamics\ZrMicro\build '
      -DCMAKE_BUILD_TYPE=Release
PS> cmake --build ZrClusterDynamics\ZrMicro\build --config Release
```

This produces `ZrClusterDynamics\ZrMicro\build\Release\solver.exe`. Use `Release`: the parameter-identification utilities call the solver thousands of times, and the OpenMP batch mode — which the coupled march relies on — is only worth having when optimized. OpenMP is detected automatically by CMake; the solver still builds and runs serially without it, and the configure output says which happened (OpenMP enabled for solver (parallel batch mode) or OpenMP NOT found).

If CMake cannot find SUNDIALS, check that the install prefix of Section 2.5 is exactly `(repo)/Libraries/sundials-7.1.1/`; that path is the first entry on the build’s `CMAKE_PREFIX_PATH`.

4.2 The 3-D MoDELib3 code (inside WSL)

MoDELib3 is built by the script `Docs/Formulation/build_modelib_wsl.sh`, which installs the required Ubuntu packages, patches the macOS-specific entries in the inherited CMake files, discards any build cache configured under a different absolute path (which is what happens when a checkout is moved), and builds the headless targets with Ninja. From the repository root:

```
PS> wsl -u root -e bash Docs/Formulation/build_modelib_wsl.sh MoDELib3
```

Note. Pass the `MoDELib3` argument explicitly, as above. The script’s no-argument default locates the `MoDELib3` checkout relative to the script’s own location assuming it sits at `ZrClusterDynamics/Docs/Formulation/`; with `Docs/` now at the repository root that default resolves outside the repository and the script stops with “`CMakeLists.txt not found`”. The explicit argument is unambiguous from any location. (Running as `-u` root avoids an interactive `sudo` password prompt for the package installs.)

The script performs, in order:

1. `apt-get install` of the toolchain and libraries: `build-essential`, `cmake`, `ninja-build`, `libeigen3-dev`, `libomp-dev`, `libfftw3-dev`, `python3-dev`, `pybind11-dev`, `libsuitesparse-dev`, `libboost-all-dev`. FFTW is *not* optional despite the CMake message that the noise generator is merely “disabled” without it: the library is linked unconditionally, and a missing FFTW fails the CMake *generate* step even though configuration succeeds.
2. Idempotent patches to the inherited `CMakeLists.txt`: the hard-coded MacPorts Eigen path is replaced by `/usr/include/eigen3`, the macOS Qt prefix path and MacPorts RPATH lines are dropped, and the Qt-only `DDqt` tool is commented out of `tools/CMakeLists.txt`.
3. Stale-cache detection: if `build/CMakeCache.txt` was configured for a different source path (a moved or copied checkout), the build tree is removed and reconfigured cleanly.
4. Configure and build: Ninja, Release, `pybind11` on, with `-DCMAKE_POLICY_VERSION_MINIMUM=3.5` so that CMake ≥ 4 accepts the repository’s legacy `cmake_minimum_required` declaration.

The build takes 10–30 minutes from clean. Its artifacts are `MoDELib3/build/tools/DDomp/DDomp` (a Linux ELF executable, invoked from Windows through `ws1.exe`) and `MoDELib3/build/tools/pyMoDELib/pyMoDELib*.so` (optional Python bindings).

4.3 Verify the builds

Re-run the resolution report of Section 3.3:

```
PS> .DisloClusterVenv\Scripts\python.exe ZrClusterDynamics\ZrMicro\py_utils\paths.py
```

The rows `0-D solver` and `MoDELib DDomp` should now read `OK`. If either is `MISS`, consult Section 9.

Warning. The governing rule of this stack: almost every failure exits zero. The `MoDELib3` build script’s final artifact check, `DDomp` with a missing initial configuration, a missing material key, a CRLF-damaged mesh, and several solver failures all return status 0. Verify by artifacts on disk and by log tails, never by exit status. The `OK/MISS` report above exists for exactly this reason.

5 First simulation: the 3-D single-crystal test

5.1 The case

`MoDELib3/tutorials/zrmicro_coupled/` is the standalone 3-D verification case, independent of the coupled march: a single cubic crystal of edge $1\ \mu\text{m}$ ($3093b$ per side, 24 115 finite-element nodes for the cluster-dynamics trial functions) with Dirichlet conditions on *all six faces*, so the entire surface acts as a grain boundary. The irradiation conditions are $T = 573\ \text{K}$, $G = 10^{-7}\ \text{dpa/s}$, zero applied stress; the run takes 31 output steps of 1 dpa each. Loop nucleation is by cascades plus homogeneous SIA clustering; the mobile species are pinned to their equilibrium values on the boundary.

Doing this case first exercises, in one pass, everything built in Sections 3 and 4: the venv, the path resolution, the WSL bridge, DDomp, and the Python post-processing.

5.2 Quick start: figures in minutes

The checkout ships with the `ev1/` output of a completed 31-dpa solve, so post-processing can be run *immediately* — no solve required:

```
PS> .DisloClusterVenv\Scripts\python.exe '
    ZrClusterDynamics\ZrMicro\py_utils\run_zr3d_singlecrystal.py '
    --doses 1 5 10 30 --max-nm 150
```

This reads `ev1/` only — it never runs a solve — and writes a timestamped run directory `ZrClusterDynamics/ZrMicro/output/<stamp>_<hash>_zr3d_ghoniem/` containing the figure sets of Table 2. The command-line options are `--doses` (which snapshots to render), `--max-nm` (depth of the grain-boundary profiles), `--tag` (suffix for the run directory), `--sim-dir` (post-process a different simulation directory), and `--no-overlays` (skip the loop-platelet overlays, which dominate the runtime).

Table 2: Output of the single-crystal post-processing.

path	contents
<code>3d/mobile_fields.png</code>	C_v , C_i , C_{2i} , C_{3i} on an interior mid-plane cut, one column per dose
<code>3d/immobile_density_fields.png</code> , <code>3d/immobile_content_fields.png</code>	the four loop families, same layout
<code>3d/loops_{c,a1,a2,a3}_overlay.png</code>	one family per figure: content field with loop platelets drawn at the true local radius, exaggerated $\times 3$ ($\langle c \rangle$) or $\times 14$ ($\langle a \rangle$)
<code>gb/gb_{density,content,size, mobile}_profiles.png</code>	radial averages over the first 150 nm from the boundary
<code>provenance.md</code>	interior state table, size-vs-dose table, and the exact command to reproduce the run

5.3 Running the solve

To regenerate `ev1/` from scratch — or to run the case at different conditions after editing the inputs (Section 8) — run the driver script inside WSL. From the repository root:

```
PS> wsl -e bash MoDELib3/tutorials/zrmicro_coupled/clean_run.sh
```

A 31-step run to 30 dpa takes about one hour on 24 cores. The script derives every path from its own location, so it works wherever the repository sits. Each invocation:

1. refreshes `inputFiles/Zr3d_ghoniem.txt` and `inputFiles/unitCube.15K.msh` from `MoDELib3/Library/`, converting line endings to LF (a CRLF mesh makes the parser return *zero nodes with no error*; a stale material file has killed runs on a missing key — hence the unconditional refresh);
2. removes and recreates `ev1/` and `F/`, then regenerates the initial configuration `ev1/ev1.0.txt` with `microstructureGenerator` (that file is both the initial configuration and the `runID-0` output, so a bare re-run would otherwise restart from the previous run’s accumulated dose);

3. runs DDomp, logging to `gen.log` and `run.log` in the case directory.

Progress can be watched from another terminal with

```
PS> wsl -e bash -c "tail -f MoDELib3/tutorials/zrmicro_coupled/run.log"
```

When the run finishes, `ev1/` holds `cdNodes.txt` (the node coordinates) and one `ev1_N.txt` per output step, and the post-processing of Section 5.2 applies unchanged.

Note. Output is written *after* each solve step, so `ev1_N` holds the state after $N+1$ dose steps: `ev1_0.txt` is 1 dpa, not 0 dpa. The dose labels passed to `--doses` account for this convention automatically.

5.4 Reading the figures critically

Two properties of the output should be kept in mind when interpreting it:

- **Platelet radii in the overlay figures are exaggerated**, and the factor differs between the $\langle c \rangle$ ($\times 3$) and $\langle a \rangle$ ($\times 14$) figures — sizes are faithful *within* a figure but not *between* figures.
- **Near-boundary loop densities are not quantitative.** The Dirichlet faces are a sink for the mobile defects but not for the loops: cascade nucleation is spatially uniform while the only loop-removal channel (coalescence) is driven by the absorbed mobile flux, which vanishes where the mobile concentrations are pinned. The $\langle a \rangle$ density therefore climbs steeply toward the boundary and grows linearly in dose there. The interior of the domain — which is what is compared against the 0-D model — is unaffected.

6 The 0-D ZrMicro simulation

6.1 Running it

Open `ZrClusterDynamics/ZrMicro/code/ZrMicro.ipynb` in JupyterLab or VS Code, select the kernel “Python 3.14 (DisloCluster)”, and run all cells. The full integration and figure suite takes minutes. Headless execution is a one-liner:

```
PS> .DisloClusterVenv\Scripts\python.exe -m nbconvert --to notebook --execute '
--ExecutePreprocessor.kernel_name=dislocluster '
--output-dir ZrClusterDynamics\ZrMicro\output '
ZrClusterDynamics\ZrMicro\code\ZrMicro.ipynb
```

The notebook builds the model chain (`InputData` $\rightarrow$ `ReactionRates` $\rightarrow$ `RateEquations`), flattens the parameter set into the `--key=value` command line that `solver.exe` consumes, integrates the 19-equation system with CVODE (BDF, dense — the recommended configuration, see Section 6.4), and writes the figure suite plus `provenance.md` to a timestamped output directory.

6.2 Inputs: the Excel workbook

All 0-D physical input lives in `ZrClusterDynamics/ZrMicro/input/Zr_input_parameters.xlsx`. Three sheets are read by the solver:

- **Material_Environment** — irradiation conditions and sink microstructure: temperature T , dose rate G , applied and hydrostatic stress, network dislocation density, grain diameter, precipitate population.

- **Physical.Properties** — lattice constants, Burgers vectors, atomic volume, formation/migration/binding energies, attempt frequencies, elastic constants, cascade efficiencies.
- **Model.Parameters** — bias factors, DAD anisotropy parameters, coalescence coefficients, nucleation sizes, dissolution parameters, obstacle strengths.

The remaining sheets hold digitized experimental data (Northwood, Gilbert, Jostsons, Cockram, Tournadre, Harte and others) used only by the offline fitting utilities (`py_utils/fit_*.py`), never by the solver.

Fitted parameters are applied *on top of* the workbook through the `PARAMETER.OVERRIDES` dictionary in the notebooks; the override always wins, and each override is routed automatically to whichever sheet already defines the key. The overrides are pinned at full precision in the notebook rather than inherited from the workbook, so that reference runs remain reproducible even as the workbook evolves (Section 10).

6.3 The state vector

The integrated state is a flat array of length 19, and the same layout is used by the Python layer, the C++ solver, and the coupled march:

Table 3: ZrMicro state vector.

slice	contents	meaning	units
<code>y[0:4]</code>	C_v, C_i, C_{2i}, C_{3i}	mobile species	atom fraction
<code>y[4:8]</code>	$C_{iL}, C_{aiL}, C_{vL}, C_{avL}$	loop number densities	per atom
<code>y[8:12]</code>	$C_{iL}^i, C_{aiL}^i, C_{vL}^v, C_{avL}^v$	loop contents	per atom
<code>y[12:18]</code>	six accumulators	conservation diagnostics	per atom
<code>y[18]</code>	ρ_N	evolving network density	m^{-2}

The four loop families are the aligned/non-aligned split of the two loop types: iL and aiL are prismatic $\langle a \rangle$ interstitial loops, vL and avL are basal $\langle c \rangle$ vacancy loops. At zero applied stress the three prismatic variants are equivalent and the aligned fraction is 1/3.

6.4 Solver configuration

The ODE backend is selected by the `solver.method` entry of the notebook’s solver configuration block:

```
'solver_method': {'backend': 'cvmde', 'lmm': 'bdf', 'linsol': 'dense'}
```

CVODE BDF with a dense linear solver is the recommended default. The accuracy benchmark against a tight-tolerance reference (at `rtol`= 10^{-6} , `atol`= 10^{-20}) is summarized in Table 4; ARKODE tables designed as IMEX pairs perform poorly in pure implicit mode on this system and should be avoided.

7 The coupled 0-D $\leftrightarrow$ 3-D simulation

7.1 The idea

The coupled march realizes the operator-split QSSA over each dose step $[\gamma_n, \gamma_{n+1}]$ in two alternating stages:

Table 4: ODE solver benchmark on the 0-D system.

config	backend	linear solver	max. rel. error
A (default)	CVODE BDF	dense	3.0×10^{-4}
B	CVODE BDF	band	3.0×10^{-4}
C	CVODE BDF	GMRES	1.7×10^{-2}
G	ARKODE SDIRK_5_3_4	dense	3.0×10^{-1}
F, H, I	ARKODE (other tables)	dense	10^4 – 10^7 (fails)

1. **Fast solve** — the steady mobile field $C_M^*(x)$ is computed with the immobile (loop) state frozen, either by the real MoDELlib3 finite-element solve or by an analytic placeholder (Section 7.4).
2. **Slow march** — the immobile ODEs are integrated independently at every quadrature point with the mobile species frozen (`freeze_mobile=1`), one OpenMP batch case per point, in a single `solver.exe` subprocess.

The loop state sets the sink field the mobile solve sees; the resulting mobile field then drives loop growth over the step. The splitting error is first order in $\Delta\gamma$. The state exchanged between the codes, and the modules that carry it, are listed in Table 5.

Table 5: The coupling contract.

direction	carrier	module
0-D $\rightarrow$ 3-D	loop density/radius per family $\rightarrow$ sink field	<code>modelib.fem.write_sink_field</code>
3-D $\rightarrow$ 0-D	steady mobile field C_M^* at quadrature points	<code>modelib.fem.solve</code>
0-D $\rightarrow$ 3-D	dose-indexed closure table	<code>modelib.export</code>
march	immobile ODE step, mobile frozen	<code>modelib.coupling.run_immobile_step</code>
both	material constants	<code>Library/Materials/Zr3d_ghoniem.txt</code>

For a gentle first contact with the coupling machinery, `code/coupling.demo.ipynb` is a minimal walk-through of one fast solve and one immobile step; the full driver described next is `code/coupled_0d_3d_ZrMicro.ipynb`.

7.2 The notebook’s three control blocks

The coupled notebook is driven entirely from three dictionaries near its top; nothing else in the notebook needs editing for a normal run.

Table 6: Control blocks of `coupled_0d_3d_ZrMicro.ipynb`.

cell	dictionary	controls
6a	GEOMETRY, MESH	domain type (<code>cubic</code> / <code>hexagonal</code>), dimensions, element size and order, boundary-layer refinement
6b	RUN	temperature, dose rate, stress, dose schedule, tolerances, substeps, march grid
6c	PARAMETER_OVERRIDES	0-D parameters applied on top of the Excel workbook

The dose schedule is `n_intervals` equal steps from `dose_seed` to `dose_max`; the defaults (1 → 26 dpa in 5 intervals) give snapshots at 1, 6, 11, 16, 21 and 26 dpa. The immobile substep count is fixed *per dose interval* by `substeps_per_interval` rather than by a wall-clock cap — a 5 dpa step at 10^{-7} dpa/s spans 5×10^7 s, which a fixed time cap would have split into a thousand batch subprocesses.

7.3 Meshing, and the materialized simulation directory

Cell 6a writes the finite-element mesh through `ZrClusterDynamics/Gmsh/generate_mesh.py` (cubic or hexagonal domains, optional boundary-layer refinement); generated `.msh` files are cached under `Gmsh/meshes/` and are reproducible from the spec, so they are not committed.

After the run directory is created, the notebook materializes a complete, self-contained MoDELlib3 simulation directory at `<run>/sim/`: the generated mesh, a deformation gradient `F` restoring the physical domain size, and a `DD.txt` whose dose schedule matches the notebook’s snapshots. **The 3-D solve itself is not launched** — the notebook prints the exact `ws1` command to run it, and the figure cells render from whichever output directory `EVL_DIR` points at (by default the shipped verification case of Section 5, so the figure cells always have data).

7.4 Fast-solve backends

The fast mobile solve inside the march is selected by `FAST_SOLVE_BACKEND`:

Table 7: Fast-solve backends.

value	behaviour
auto (default)	MoDELlib3 FEM if <code>DDomp</code> is built <i>and</i> the simulation directory is seeded, else the placeholder
modelib	require the real FEM solve; raise if unavailable
placeholder	analytic $C_M^*(x) = [1 - e^{-x/\ell}] C_M^{\text{bulk}}$ — pipeline checks only

Warning. The placeholder is *not* a MoDELlib3 result. It scales all four mobile species by the same factor, which cannot represent species-dependent depletion lengths that differ by four orders of magnitude; its boundary profile is an artifact and must not be read physically. The notebook states loudly in its output which backend is active — read that message every time. To activate the real backend, seed a simulation directory once:

```
from py_utils.modelib_fem import seed_sim_dir_from_tutorial
from py_utils import paths
seed_sim_dir_from_tutorial(paths.MODELIB_ROOT, paths.OUTPUT_DIR / "modelib_sim")
```

then run its `generateInputFiles.py` with MoDELlib3’s `MicrostructureGenerator`. Until that is done, `auto` silently falls back to the placeholder.

7.5 Running the march

Interactively: open `coupled_0d_3d_ZrMicro.ipynb` on the “Python 3.14 (DisloCluster)” kernel and run all cells. Headless:

```
PS> .DisloClusterVenv\Scripts\python.exe -m nbconvert --to notebook --execute ‘
```

```
--ExecutePreprocessor.kernel_name=disloccluster '
--output-dir ZrClusterDynamics\ZrMicro\output '
ZrClusterDynamics\ZrMicro\code\coupled_0d_3d_ZrMicro.ipynb
```

Before the march begins, the notebook (i) ensures both builds — `ensure_zrmicro_solver()` configures and builds `solver.exe` through CMake if it is absent, and `ensure_modelib()` checks for DDomp, invoking the WSL build script if necessary; both verify artifacts on disk rather than trusting exit status — (ii) runs the full 0-D reference integration, which supplies the physical reference and the initial immobile state, and (iii) performs a field-by-field consistency check between the MoDELlib3 material file and the 0-D conditions, reporting both the matches and the *known* non-correspondences (Appendix B) so that a residual is not mistaken for a bug.

7.6 Outputs

Every coupled run writes `output/(YYYYMMDD_HHMMSS)_(git-hash)_coupled0d3d/` containing `march_state.npz` (the full march state), `provenance.md`, and three figure directories:

Table 8: Coupled-run figure output.

directory	contents
0d/	the 0-D figure suite, unchanged
3d/	<code>fe_mesh.png</code> , plus one file per quantity per dose: <code>Cv_06dpa.png</code> , <code>N_c_06dpa.png</code> , <code>C_a1_06dpa.png</code> , <code>loops_c_06dpa.png</code> , ...
gb/	one file per quantity with all doses overlaid (<code>gb-Cv.png</code> , <code>gb-N_a1.png</code> , <code>gb-C_c.png</code> , <code>gb-d.c.png</code>), plus <code>size_dist_(family)-(dose).png</code>

Two interpretation caveats, in addition to those of Section 5: the loop-overlay figures here fill the domain with non-overlapping platelets at radii exaggerated $\times 1.5$ ($\langle c \rangle$) and $\times 7$ ($\langle a \rangle$), and they are the slow figures (~ 40 s each). The `size_dist.*` figures show the distribution of the *local mean* loop diameter across the domain, weighted by local loop density — not a per-loop size spectrum; the model carries one mean size per family per node, so spatial variation is the only source of spread.

8 Changing things

Table 9 maps the common modifications onto the files and control blocks that own them.

Three of these deserve elaboration.

8.0.0.1 Temperature appears in three places. The workbook, the notebook’s RUN block, and `polycrystal.txt` each carry a temperature, and they are not linked. Moreover the DAD reconciliation parameters `dadAnisotropy` (p_m) and `dadZ0` (Z_m^0) in `Zr3d_ghoniem.txt` are *fitted at* $T = 573$ K by closed-form inversion against the 0-D symmetric capture efficiencies. Changing the temperature requires refitting them, or the DAD bias will be silently wrong.

8.0.0.2 The 3-D dose step is expressed in MoDELlib3 time units. In `DD.txt`, `dtMax` = 6.95869×10^{19} corresponds to 1 dpa at $G = 10^{-7}$ dpa/s (the MoDELlib3 time unit is $b/c_s = 1.437 \times 10^{-13}$ s; see the unit table in Appendix A). Scale it proportionally when changing the dose increment or the dose rate.

Table 9: Where to change what.

to change	edit
irradiation conditions (T, G, σ)	the RUN dictionary of the coupled notebook <i>and</i> <code>inputFiles/polycrystal.txt</code> / <code>Zr3d_ghoniem.txt</code> for the 3-D case; they are not linked
0-D material parameters	the workbook <code>input/Zr_input_parameters.xlsx</code> , or <code>PARAMETER.OVERRIDES</code> (which wins)
3-D material parameters	<code>MoDELib3/Library/Materials/Zr3d_ghoniem.txt</code> , then re-run <code>clean_run.sh</code> so the copy in <code>inputFiles/</code> is refreshed
dose schedule (coupled march)	<code>dose_seed</code> , <code>dose_max</code> , <code>n_intervals</code> , <code>substeps_per_interval</code> in RUN
dose schedule (3-D single crystal)	<code>Nsteps</code> and <code>dtMax</code> in <code>tutorials/zrmicro.coupled/inputFiles/DD.txt</code>
mesh or domain (coupled march)	<code>GEOMETRY</code> / <code>MESH</code> in cell 6a (regenerates through Gmsh)
mesh or domain (3-D single crystal)	<code>meshFile</code> and the deformation gradient <code>F</code> in <code>inputFiles/polycrystal.txt</code>
0-D solver method	<code>solver_method</code> in the solver configuration (Section 6.4)

8.0.0.3 Keep `startAtTimeStep` at zero. Never set it to -1 : that silently restarts from a stale `ev1_N` and double-counts dose. `clean_run.sh` pins it and regenerates `ev1.0.txt` for exactly this reason.

9 Troubleshooting

The single most important habit was already stated in Section 4: *almost every failure in this stack exits zero*, so verify by artifacts on disk (the OK/MISS report, log tails, file sizes) rather than by exit status. Table 10 collects the known symptoms.

Table 10: Symptoms and causes.

symptom	cause and fix
<code>paths.py</code> reports MISS for a location that exists	The checkout deviates from the standard layout. Set <code>DISLOCLUSTER.ROOT</code> , <code>MODELIB.ROOT</code> and/or <code>MODELIB.BUILD</code> explicitly and re-run the report.
<code>pip install</code> fails or SciPy import errors	Anaconda Python was used to create the venv. Recreate it from the python.org CPython 3.14 (Section 3).
CMake cannot find SUNDIALS	The install prefix is not <code>(repo)/Libraries/sundials-7.1.1/</code> and no system install is registered. Redo Section 2.5.
MoDELib3 CMake <i>generate</i> step fails after a successful configure	<code>libfftw3-dev</code> is missing — the library is linked unconditionally despite the “noise generator disabled” message. The build script installs it; if building by hand, install it first.
MoDELib3 reconfigure hard-errors: “the current CMakeCache.txt directory ... is different”	The checkout was moved or copied, so the cached absolute paths are stale. <code>build_modelib_ws1.sh</code> detects and discards the stale cache; by hand, delete the <code>build/</code> tree and re-configure. The same applies to <code>ZrMicro/build/</code> on the 0-D side.

symptom	cause and fix
<code>build_modelib_wsl.sh</code> stops with “CMakeLists.txt not found”	The script was run without the explicit <code>MoDELib3</code> argument (Section 4.2).
<code>microstructureGenerator</code> never terminates	It is running the discrete-dislocation path, inserting loops one at a time; continuum loop densities would need $\sim 10^{11}$ of them. Set <code>useDislocations=0</code> and zero target densities — the shipped <code>inputFiles/</code> already do.
Parser reports zero mesh nodes, no error	CRLF line endings in the <code>.msh</code> . <code>clean_run.sh</code> LF-converts it on every run for this reason.
Run restarts from the previous dose	<code>ev1.0.txt</code> is both the initial configuration and the <code>runID-0</code> output. Regenerate it every run (<code>clean_run.sh</code> does).
Loop content comes out at exactly $3\times$ the expected value	<code>startAtTimeStep=-1</code> resumed from a stale <code>ev1_N</code> and double-counted dose. Pin <code>startAtTimeStep=0</code> .
Newton residual stalls near 10^5	An initial-guess problem, not a linear-algebra one; the iterative (BiCGSTAB) branch is the working one on the $\sim 96k$ -dof system.
Notebook silently produces analytic mobile profiles	The fast-solve backend fell back to <code>placeholder</code> . Check that <code>DDomp</code> is built and the simulation directory is seeded (Section 7.4).
C_{2i} , C_{3i} orders of magnitude low	Historically a member-initialization-order bug in <code>ClusterDynamicsParameters</code> (fixed on this branch; see Appendix B). If it reappears, check that no member’s initializer reads a member declared after it.
Three $\langle a \rangle$ families give different densities at zero stress	Burgers vectors supplied in Cartesian rather than lattice components. The zero-stress symmetry check is the fastest detector of this error class.
Notebook regression cell reports “skipped”	The pinned 0-D baseline run directory is not present in this checkout. Point <code>REFERENCE_RUN</code> at any local run directory to re-enable the regression.

10 Reproducibility and provenance

Every run — 0-D, 3-D post-processing, or coupled — writes a timestamped, git-tagged directory

`ZrClusterDynamics/ZrMicro/output/<YYYYMMDD_HHMMSS>_<git-hash>[_<tag>]/`

containing the figures and `provenance.md`, which records the resolved repository layout, every input parameter, the active fast-solve backend (for coupled runs), the software environment, and the solver statistics — so every figure and every number is traceable to a commit. Because the repository root may not itself be a git repository, `paths.git_hash()` falls back through `root` → `ZrClusterDynamics` → `MoDELib3`, so the tag stays meaningful.

Warning. `provenance.md` stores values to four significant figures only, so re-reading it will *not* reproduce a run bit-for-bit. The full-precision fitted parameter set lives in the `PARAMETER_OVERRIDES` block of the notebooks, which is why that block is pinned rather than inherited from the workbook.

A Architecture reference

This appendix condenses the full architecture description (the companion document *Zr3d Code Architecture*, `Docs/Formulation/`) to what a user needs when reading code or diagnosing a run.

A.1 The governing split

The mobile species are posed as a quasi-steady reaction–diffusion boundary-value problem with no time derivative,

$$0 = -\nabla \cdot \mathbf{J} + P_M + \frac{1}{2} \mathbf{c}_M^T R_2 \mathbf{c}_M + (E - D) \mathbf{c}_M + S_{vL}, \quad (1)$$

and the immobile loop families are marched pointwise in dose against the frozen mobile field. Both execution routes realize this split:

- **Route A — notebook-driven coupled march** (Section 7). Python owns the dose loop; MoDELib3 solves the fast mobile BVP; the ZrMicro C++ integrator advances the immobile state with `freeze_mobile=1`, one OpenMP batch case per quadrature point.
- **Route B — monolithic MoDELib3 run** (Section 5). Everything happens inside DDomp: `ClusterDynamicsFEM::solveMobileClusters()` performs the Newton solve of Eq. (1) and `solveImmobileClusters()` marches the loop families node by node. No Python at run time.

A.2 Key components

A.3 The three interfaces

All cross-component state flows through three deliberately narrow interfaces:

1. **Python → ZrMicro C++:** a flat `--key=value` command line, one token per scalar; results return as a whitespace-delimited matrix on stdout. In batch mode, one case per line of the batch file, each output block prefixed `=== CASE <i> status=<s> ===`. Purely textual — no build-time coupling, so either side can be rebuilt independently.
2. **Python ↔ MoDELib3:** the file system. `write_sink_field()` writes the loop state into `inputFiles/aLoopsDensity.txt` and `inputFiles/frankLoopsDensity.txt`; DDomp runs on the directory; the mobile field is read back from its output. On Windows this crosses the WSL boundary, with Windows paths translated to `/mnt/<drive>/...`
3. **MoDELib3 → post-processing:** `ev1/ev1_N.txt` (12 columns per node: four mobile species, four loop densities, four loop contents) plus `ev1/cdNodes.txt` (the node coordinates). The second file is essential: the cluster-dynamics trial functions live on second-order elements, so the field rows do *not* correspond to the mesh vertices and the ordering cannot be reconstructed externally.

A.4 Unit systems

The two codes do not share a unit system, and most historical reconciliation defects were unit errors.

With $\Omega_{b3} = \Omega/b^3 = 0.355105$, the density conversion is $N_{0D} = N_{3D} \Omega_{b3}$, and the defects per loop follow as $m_k = c_k/(N_k \Omega)$. One dose step of 1 dpa at $G = 10^{-7}$ dpa s⁻¹ is 6.95869×10^{19} MoDELib3 time units.

Table 11: The components a user is most likely to touch. Route marks where each participates: A = coupled march, B = monolithic run, P = post-processing.

component	route	role
py_utils/paths.py	A,B,P	canonical path resolution (Section 1.3)
py_utils/input_data.py	A	reads the workbook, computes every derived quantity
py_utils/reaction_rates.py, rate.equations.py	A	the 0-D physics; deliberate mirror of the C++ RHS
py_utils/cpp_bridge.py	A	flattens parameters to the --key=value CLI, launches solver.exe, parses stdout
py_utils/modelib_coupling.py	A	the operator-split driver (pack_y0, run_immobile_step)
py_utils/modelib_fem.py	A	adapter to MoDELib3: writes the sink field, invokes DDomp (through wsl.exe when required), reads the mobile field back
py_utils/modelib_export.py	A	maps the lumped 0-D loop families onto the three $\langle a \rangle$ prismatic variants
py_utils/modelib_report.py, modelib_fields.py, modelib_gb.py	B,P	read evl/, render field panels, overlays and grain-boundary profiles
py_utils/run_zr3d_singlecrystal.py	B,P	the single-crystal post-processing driver (Section 5)
code/solver.cpp + cpp_utils/	A	the 0-D integrator: CVODE/ARKODE, OpenMP batch mode, one SUNContext per thread
MoDELib3/tools/DDomp/	B	the 3-D runner
ClusterDynamics(FEM,Parameters)	B	the 3-D numerical core: weak forms, mobile Newton iteration, immobile march

Table 12: Unit conventions.

quantity	ZrMicro	MoDELib3
length	m	$b = 3.233 \times 10^{-10}$ m
time	s	$b/c_s = 1.437052 \times 10^{-13}$ s
mobile concentration	atom fraction	atom fraction (same)
loop number density	per atom	per b^3
loop content	per atom	volume fraction (numerically the same)

A.5 Call graph

Figure 1 is the component call graph across both routes.

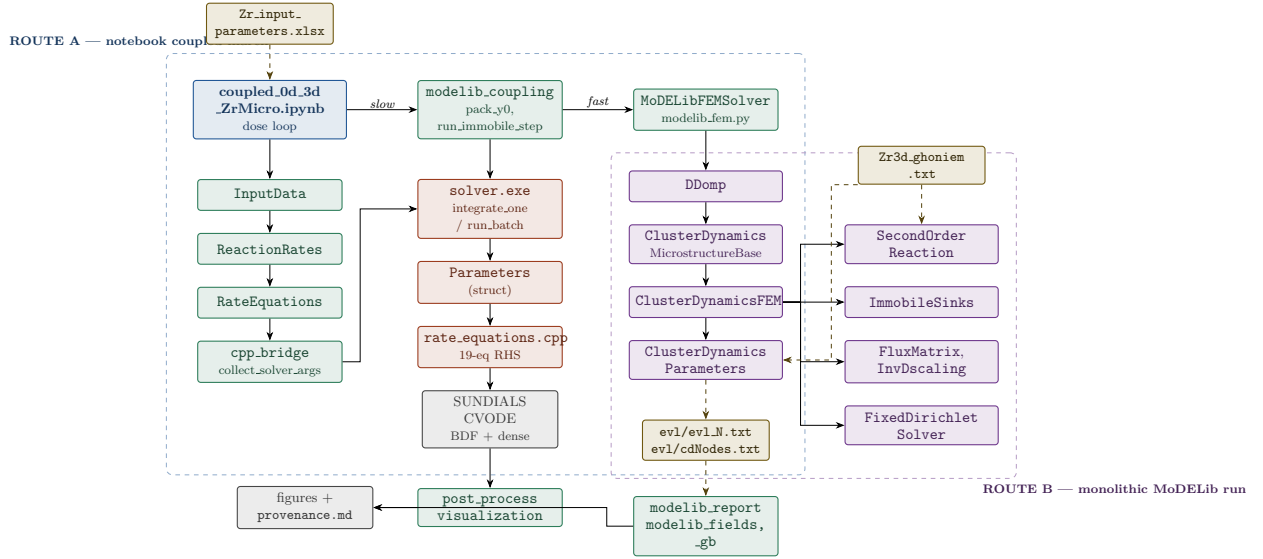


Figure 1: Component call graph. Solid arrows are calls, dashed arrows are file input/output. Blue: orchestrating notebook; green: ZrMicro Python; red: ZrMicro C++; purple: MoDELib3 C++; ochre: files on disk; gray: products. Route A drives the fast mobile solve through MoDELib3 and the slow immobile march through the ZrMicro integrator; Route B runs both inside MoDELib3. The two routes share the material definitions and the post-processing layer.

A.6 Known non-correspondences between the 0-D and 3-D models

These are physics, not bugs — do not “fix” them silently:

1. **DAD bias.** The 0-D uses phenomenological anisotropy parameters δ_i, δ_v with $Z_a + Z_c = 2$; the 3-D generates the bias from the diffusion tensors via p_m . They are reconciled by a closed-form fit of (Z_m^0, p_m) in `Zr3d_ghoniem.txt`, valid only at the fitted temperature. `Zr4.txt` is the standalone 3-D calibration and carries no `dad*` keys — do not check against it.
2. **Bi-pyramid vacancy family** — exists only in the 3-D model.
3. **Size-dependent vacancy-loop thermal emission** — 3-D only; the 0-D uses a constant surrogate.

B Changes relative to upstream MoDELib2-NNL

MoDELib3 is a fork of `m1m335/MoDELib2-NNL`. The upstream code solves the mobile steady-state PDE of Eq. (1) — and nothing else: on upstream `main` the immobile species count is zero, `solveImmobileClusters()` is an empty function body, and `ImmobileSinks.h` does not exist. The entire immobile half of the model was added in this fork:

- the loop density and content evolution equations (`solveImmobileClusters()`);

- homogeneous SIA-clustering nucleation, read off the reaction network the mobile solve already uses (`getLoopNucChannels()`);
- the loop absorption sink on the mobile species (`ImmobileSinks.h`), deliberately excluding the grain boundary, which the spatially resolved model captures through the Dirichlet condition and would otherwise be counted twice;
- DAD capture efficiencies from (p_m, Z_m^0) (`loopDADbias()`);
- absorption fluxes with cluster weighting, the minimum-stable-size gate, vacancy-loop dissolution, size-dependent thermal vacancy emission, and two-channel Avrami coalescence.

Four defects were also corrected in pre-existing upstream code: a member-initialization-order bug that silently dropped the Boltzmann factor from the cluster dissociation rates; a spurious scaling of the diffusion operator relative to the reaction terms after the cluster-dynamics atomic volume became independently settable; a unit error that left the minimum-stable-radius gate permanently open; and the use of the two-atom hexagonal lattice *cell* volume where the atomic volume was intended (now settable explicitly via the optional `atomicVolume_SI` material key).

On the input side, Po’s standalone calibration `Zr4.txt` is left untouched; the coupled, reconciled definition is the separate file `Zr3d_ghoniem.txt`, which resets the mobile block (cascade fractions, migration energies, diffusivities, reaction prefactors, network sink) to correspond with the 0-D model and adds the entire immobile block (~ 20 new keys). The full before-and-after, key by key, is in `MoDELib3/ZR3D_GHONIEM_CHANGES.md` and in the companion architecture document.

C Quick reference

The complete sequence from a fresh Windows machine to figures, assuming the prerequisites of Section 2 (Git, Python 3.14, VS Build Tools, CMake, WSL + Ubuntu, SUNDIALS in `Libraries/`) are in place. All commands from the repository root.

```
# 1. clone
PS> git clone https://github.com/Ghoniem/DisloCluster.git ; cd DisloCluster

# 2. python environment + kernel
PS> C:\Python314\python.exe -m venv .DisloClusterVenv
PS> .DisloClusterVenv\Scripts\python.exe -m pip install -r requirements.txt
PS> .DisloClusterVenv\Scripts\python.exe -m ipykernel install --user '
    --name dislocluster --display-name "Python 3.14 (DisloCluster)"

# 3. verify path resolution (repeat after each build)
PS> .DisloClusterVenv\Scripts\python.exe ZrClusterDynamics\ZrMicro\py_utils\paths.py

# 4. build the 0-D solver (native Windows)
PS> cmake -S ZrClusterDynamics\ZrMicro\cpp_utils -B ZrClusterDynamics\ZrMicro\build '
    -DCMAKE_BUILD_TYPE=Release
PS> cmake --build ZrClusterDynamics\ZrMicro\build --config Release

# 5. build the 3-D code (inside WSL; note the explicit MoDELib3 argument)
PS> wsl -u root -e bash Docs/Formulation/build_modelib_wsl.sh MoDELib3

# 6. first test: figures from the shipped 3-D results (minutes)
PS> .DisloClusterVenv\Scripts\python.exe '
    ZrClusterDynamics\ZrMicro\py_utils\run_zr3d_singlecrystal.py '
    --doses 1 5 10 30 --max-nm 150
```

```

# 7. the full 3-D solve (hours) -- then rerun step 6
PS> wsl -e bash MoDELib3/tutorials/zrmicro_coupled/clean_run.sh

# 8. the 0-D model (minutes)
PS> .DisloClusterVenv\Scripts\python.exe -m nbconvert --to notebook --execute '
--ExecutePreprocessor.kernel_name=dislocluster '
--output-dir ZrClusterDynamics\ZrMicro\output '
ZrClusterDynamics\ZrMicro\code\ZrMicro.ipynb

# 9. the coupled 0-D <-> 3-D march (open interactively, or:)
PS> .DisloClusterVenv\Scripts\python.exe -m nbconvert --to notebook --execute '
--ExecutePreprocessor.kernel_name=dislocluster '
--output-dir ZrClusterDynamics\ZrMicro\output '
ZrClusterDynamics\ZrMicro\code\coupled_0d_3d_ZrMicro.ipynb

```

Key locations:

what	where
path resolution (run it!)	ZrClusterDynamics/ZrMicro/py_utils/paths.py
0-D inputs	ZrClusterDynamics/ZrMicro/input/Zr_input_parameters.xlsx
3-D material file	MoDELib3/Library/Materials/Zr3d_ghoniem.txt
3-D case inputs	MoDELib3/tutorials/zrmicro_coupled/inputFiles/
all outputs	ZrClusterDynamics/ZrMicro/output/
env overrides	DISLOCLUSTER_ROOT, MODELIB_ROOT, MODELIB_BUILD